

Visual Datacenter — Technical Design

1. Architectural Styles

The system is designed around a **decoupled Client-Server (REST API)** architectural style.

- **Frontend SPA:** A single-page React application built with TypeScript, Vite, and TailwindCSS, utilizing Three.js (via React Three Fiber) for 3D room rendering and canvas/DOM elements for 2D rack inspection.
 - **Backend API:** A Spring Boot API (Java 25) acting as a stateless REST service that exposes resources under `/api/v1`.
 - **Stateless REST Communication:** Data transfer occurs via JSON payloads. The backend is the authoritative source of validation and state management.
 - **Sandbox Model:** The initial system operates as a single-user sandbox without authentication or role-based access control, allowing rapid iteration of visual tools.
-

2. C4 Diagrams

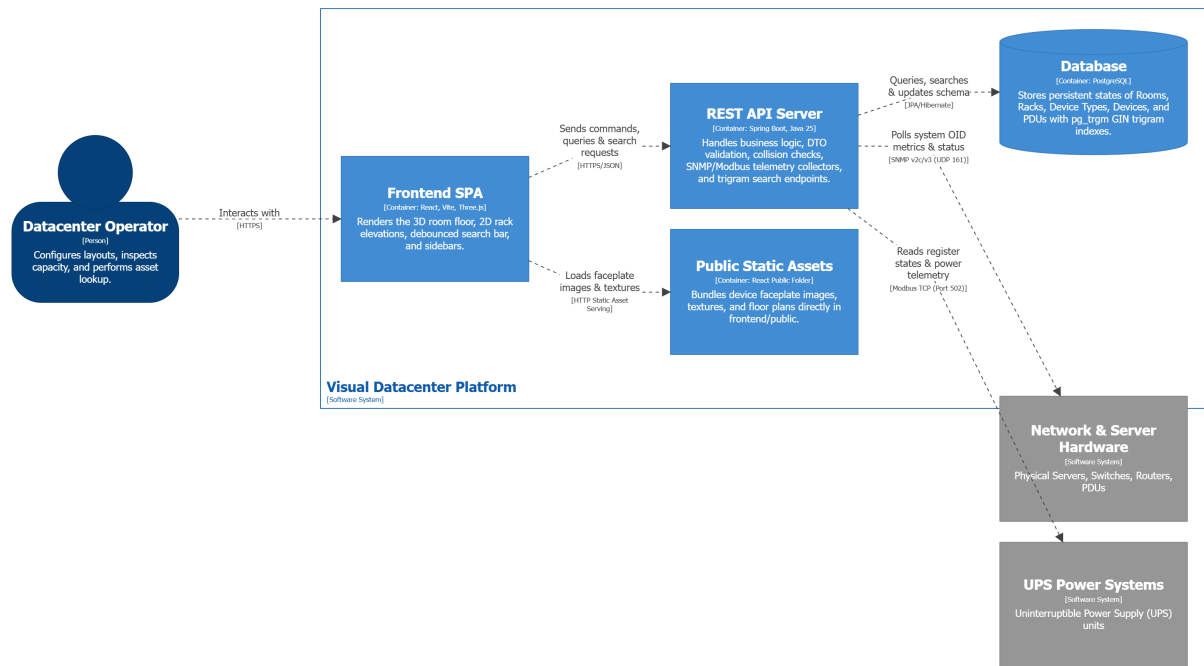
Level 1 — System Context

The system context diagram shows the user (Datacenter Operations Manager/Systems Engineer) interacting with the Visual Datacenter platform and external physical equipment.



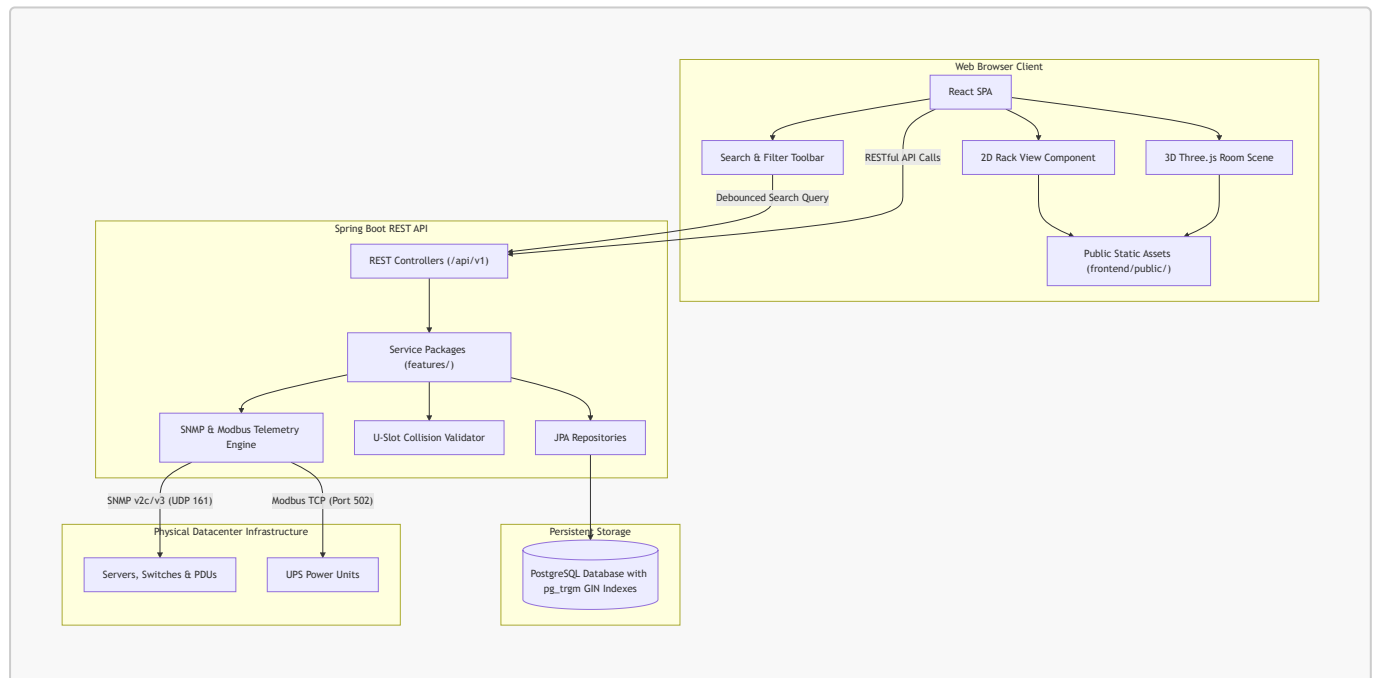
Level 2 — Containers

The container diagram outlines the division between the React Frontend SPA, the Spring Boot Backend API, the PostgreSQL database, frontend static assets, and physical hardware communication endpoints.



3. High-Level Architecture

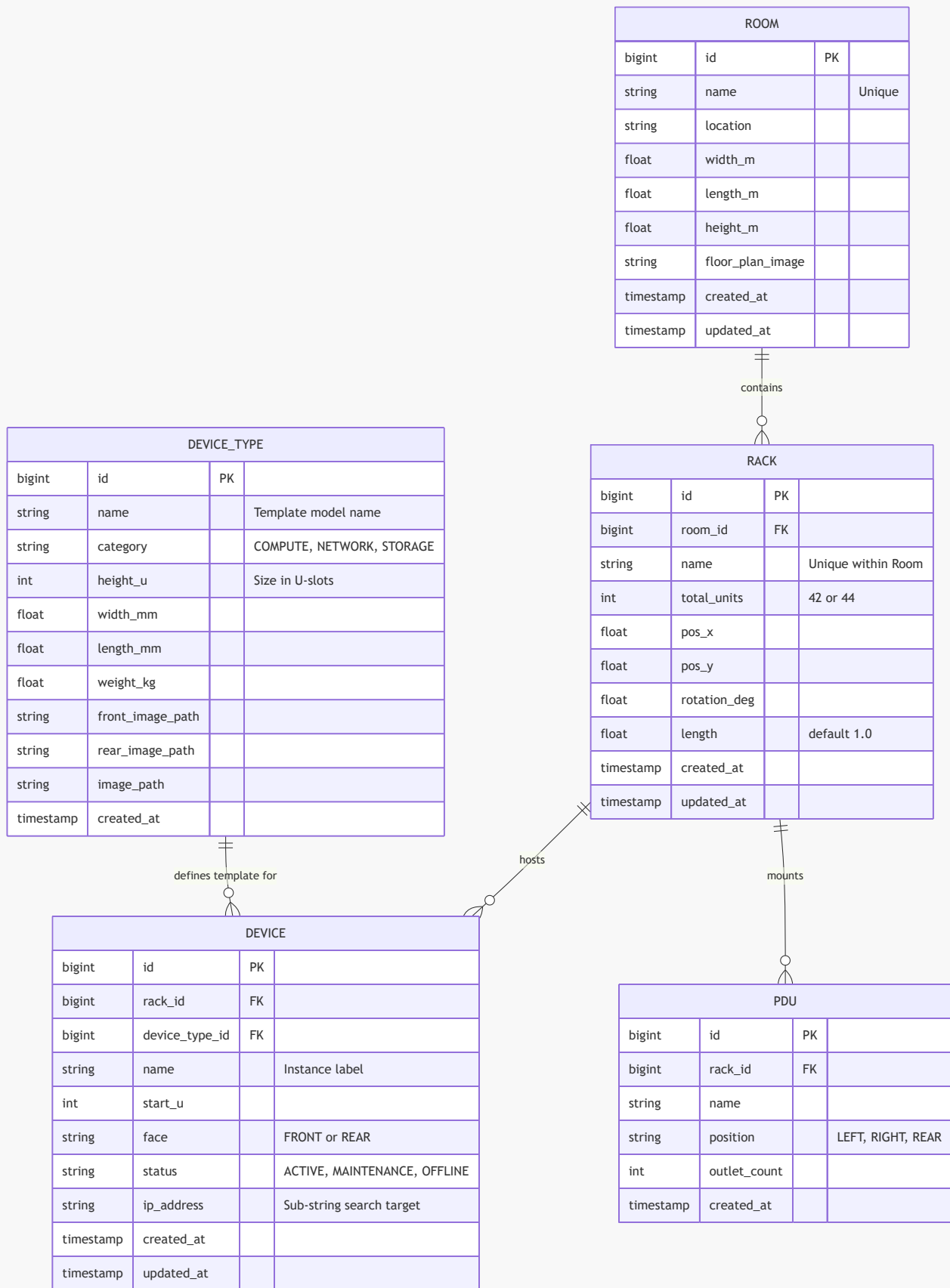
The components flow from the browser interfaces to the Spring Boot feature packages, the database, and physical hardware equipment:



4. Database Schema Logic

Entity Relationship Diagram (ERD)

The database schema defines the relationships between rooms, racks, devices, PDUs, and device types.



Core Invariants

Entity	Core Validation Rules & Invariants
Room	$\text{width_m} > 0, \text{length_m} > 0$; System-wide unique name.
Rack	$\text{total_units} \in \{42, 44\}$; Coordinates pos_x and pos_y must be within Room boundaries; Unique name within Room. Color-coded in 3D based on U-slot utilization (High $\geq 80\%$ Red, Medium $\geq 50\%$ Yellow, Low $< 50\%$ Green) across standard, selected highlight, and X-ray mode outer-frame states.
Device	$\text{start_u} \geq 1$ and $\text{start_u} + \text{device_type.height_u} - 1 \leq \text{rack.total_units}$. Overlap validation (U-slot collision detection).
PDU	$\text{position} \in \{\text{LEFT}, \text{RIGHT}, \text{REAR}\}$; maximum 3 PDUs attached to a single rack (one per position).
Device Type	$\text{height_u} \geq 1$, width and length positive; deletion blocked if referenced by active device instances.

U-Slot Collision Detection Logic

Before persisting any device installation, the service layer enforces the following transactional logic:

1. **Define Target Interval:** A device of height H installed at start unit S occupies slots:

$$I_{\text{target}} = [S, S + H - 1]$$

2. **Fetch Existing Intervals:** Query all devices currently installed in the same rack ID where face matches the target installation face (e.g. FRONT or REAR).
3. **Conflict Check:** For each existing device with interval $I_{\text{existing}} = [E_{\text{start}}, E_{\text{start}} + E_{\text{height}} - 1]$:

$$\text{Conflict} \iff I_{\text{target}} \cap I_{\text{existing}} \neq \emptyset$$

$$\text{Conflict} \iff \max(S, E_{\text{start}}) \leq \min(S + H - 1, E_{\text{start}} + E_{\text{height}} - 1)$$

4. **Action:** If a conflict is detected, abort the transaction and return HTTP 409 Conflict. If no conflicts exist, proceed with persistence.

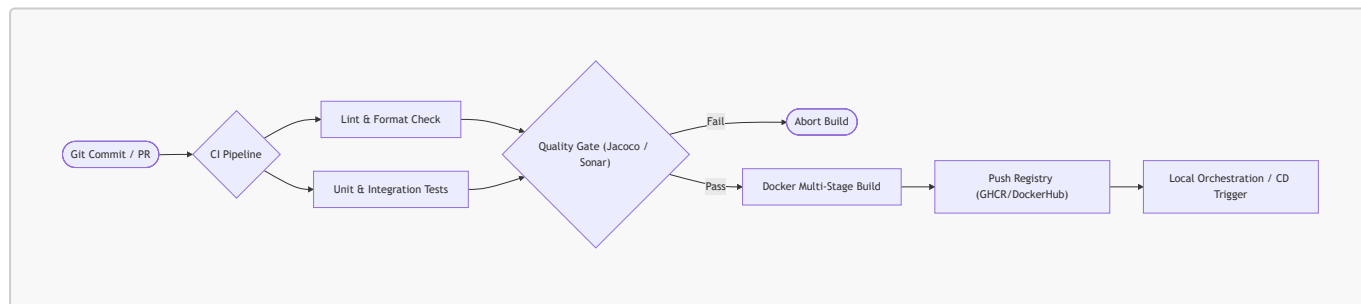
5. Architecture Decision Records (ADRs)

ADR #	Decision	Context / Rationale	Consequences
ADR-01	Decoupled Spring Boot + React	Separates business domain logic from visualization and client-side interactions.	Clean API contract; separate build steps.
ADR-02	Three.js for 3D Room View	High performance, rich WebGL libraries, and robust OrbitControls / raycasting support.	High dependency on browser WebGL support.
ADR-03	Instanced Mesh for 10k+ Devices	To maintain FPS > 40, rendering individual unique BoxGeometries for thousands of devices causes draw-call bottlenecks. Instanced meshes share geometry and materials, executing in a single draw-call.	Requires centralizing mesh updates and tracking instance index offsets.

ADR #	Decision	Context / Rationale	Consequences
ADR-04	Authoritative Server-side Collision	Client-side layout tools are optimistic for speed, but the database integrity relies on Spring Boot's service validations.	Slightly higher API overhead during device installation.
ADR-05	No Authentication in MVP	Allows zero-friction sandbox usage.	Security checks and login screens must be implemented in a subsequent deployment phase.
ADR-06	Frontend Public Asset Bundling	Serve faceplate and texture images directly from the React static public directory (<code>frontend/public/</code>).	Eliminates backend file I/O overhead and complex file upload handling; assets are bundled with the web client.
ADR-07	Dual-Face (Front/Rear) Texture Mapping	3D device materials map to front face, rear face and while 2D rack elevation renders the active faceplate.	Ensures accurate hardware visualization regardless of installation orientation.

6. CI/CD Pipeline Design

The system implements automated builds, testing, and container packaging using a unified GitHub Actions pipeline.



7. Quality Gates

To guarantee development speed without sacrificing runtime stability (FPS > 40 under high load), code must pass the following quality thresholds before being merged into `main` or `develop`:

Static Analysis and Code Rules

- **0 Blocker / Critical Bugs:** No warning labels or safety issues detected by SonarQube.
- **Zero Syntax & Linter Errors:** ESLint configurations must be completely clean on the frontend; Checkstyle/Spotless checks must compile warning-free on the backend.
- **Complexity Thresholds:** Cognitive complexity of React rendering components must remain low; complex 3D computations (like instanced mesh coordinates) must be isolated into helper functions with

dedicated unit tests.

Test Coverage Requirements

- **Backend Test Coverage:** Minimum **80% line and branch coverage** verified by Jacoco. Focus on validating U-slot collision boundaries and room coordinate validations.

Container Security

- **Vulnerability Scan:** Docker images undergo standard automated vulnerability scans using Snyk during the build stage.